

Ahime - Préparation à la publication

Play Store & App Store - Résumé des travaux

Généré le 20 août 2026 - Projet Flutter Ahime (c:\Users\NKOUASSI\Documents\GitHub\Ahime-main)

1. Vue d'ensemble

Le projet Flutter Ahime a été préparé pour la soumission sur le Google Play Store et l'Apple App Store. Les blocages techniques identifiés (identifiant d'application générique, absence de signature de release, icônes/splash par défaut, non-conformité aux exigences récentes de Google, textes de permission placeholder côté iOS) ont été corrigés. Le tout a été committé et poussé sur la branche main du dépôt GitHub.

2. État par plateforme

Application ID / Bundle ID (com.ahime.app)	TERMINE
Signature de release Android (keystore)	TERMINE
Minification R8 / ProGuard	TERMINE
Conformite "16 KB page size" (Gradle/AGP/Kotlin)	TERMINE
Icones adaptatives Android + jeu d'icônes iOS	TERMINE
Splash screen natif (clair/sombre)	TERMINE
Permissions iOS (descriptions réelles)	TERMINE
App Bundle (.aab) genere et teste	TERMINE
Compilation/signature iOS (necessite un Mac)	A FAIRE
Comptes Google Play Console / Apple Developer	A CREER
Politique de confidentialite hebergee	A COMPLETER
Fiches store (captures, description...)	A PREPARER

3. Android - détail

Identité de l'application

- applicationId / namespace : com.example.ahime -> com.ahime.app
- Package Kotlin déplacé vers android/app/src/main/kotlin/com/ahime/app/MainActivity.kt

Signature de release

- Nouveau keystore généré (RSA 2048, validité jusqu'en 2054, alias "ahime")
- Configuré via android/key.properties (exclu de git)
- APK et App Bundle vérifiés signés avec le certificat de release (SHA-256 confirmé via apksigner)

Corrections de conformité

- Gradle 8.3 -> 8.9, Android Gradle Plugin 8.2.1 -> 8.7.3, Kotlin 1.8.22 -> 2.1.0
- Corrige l'alignement mémoire "16 KB page size" exigé par Google Play depuis novembre 2025
- Minification R8 et shrinkResources activés pour le build release, avec proguard-rules.pro

Icônes et splash

- Icône adaptative Android 12+ générée (mipmap-anydpi-v26) à partir du logo Ahime
- Splash natif refait : fond blanc / fond sombre, logo transparent centré

4. iOS - détail

Identité de l'application

- Bundle ID : com.example.ahime -> com.ahime.app (Runner + RunnerTests)

Corrections critiques pour la revue Apple

- Les descriptions de permission de localisation dans Info.plist contenaient le texte placeholder "..." - motif de rejet automatique par Apple. Remplacées par de vrais textes explicatifs en français.
- Ajout de descriptions caméra/photothèque (utilisées potentiellement par les WebView intégrées)
- Jeu complet d'icônes iOS régénéré à partir du logo Ahime

Limite technique

La compilation, la signature et l'archivage de l'application iOS nécessitent Xcode, qui ne fonctionne que sur macOS. Cette étape ne peut pas être réalisée depuis cet environnement Windows - elle devra être faite sur un Mac (ou via un service de build cloud type Codemagic/Fastlane) avant l'upload sur App Store Connect.

5. Nettoyage du dépôt

Un commit local non poussé contenait par erreur environ 200 Mo de fichiers (cache complet d'un profil Chrome headless + une archive Gradle), dépassant la limite GitHub de 100 Mo et bloquant silencieusement tout push. Comme ce commit n'avait jamais atteint le dépôt distant, il a pu être fusionné proprement avec les changements de cette session, sans réécrire d'historique déjà partagé ni recourir à un force-push.

- Fichier Gradle de 200 Mo retiré de l'historique local
- .dart_tool/chrome-device/, .dart_tool/flutter_build/, .dart_tool/pub/ et build/ retirés du suivi git
- .gitignore mis à jour pour empêcher toute récurrence
- Push final effectué en fast-forward sur origin/main (commit f2fb3f8)

6. Sécurité - à conserver précieusement

Le keystore de signature Android et ses identifiants ne sont volontairement pas reproduits dans ce document. Ils se trouvent uniquement en local sur cette machine :

```
C:\Users\NKOUEASSI\keystores\ahime-release-key.jks  
C:\Users\NKOUEASSI\keystores\ahime-release-CREDENTIALS-KEEP-SAFE.txt
```

Ce keystore est indispensable pour publier toute future mise à jour de l'application sur le Play Store. En cas de perte, il est impossible de mettre à jour l'application existante. Il est fortement recommandé d'en faire une copie de sauvegarde chiffrée (gestionnaire de mots de passe, coffre-fort numérique) en dehors de cette machine.

7. Prochaines étapes

- Créer un compte Google Play Console (frais unique de 25 \$)
- Créer un compte Apple Developer Program (99 \$/an)
- Compléter et héberger PRIVACY_POLICY.md sur une URL publique (obligatoire, géolocalisation utilisée)
- Compiler et signer l'app iOS sur un Mac, puis uploader via Xcode ou Transporter
- Uploader build/app/outputs/bundle/release/app-release.aab sur Google Play Console
- Préparer les fiches store : captures d'écran, description, catégorie, classification d'âge

8. Fichiers livrés

build/app/outputs/bundle/release/app-release.aab	(Play Store)
build/app/outputs/flutter-apk/app-release.apk	(test uniquement)
PRIVACY_POLICY.md	(brouillon à compléter)